Gbar 开发主手册

> 本文档是 Gbar 项目的核心开发手册，记录关键约定、接口扩展流程与组件索引。
> 架构细节与待办事项请查看 docs/tip。

目录

1. 项目约定
2. IGameContext 扩展流程
3. 消息队列使用约定
4. 系统生命周期
5. ECS 组件索引
6. 相关文档

项目约定

约定	说明
ECS 权威数据源	ECS 持有最终位置、状态、属性；物理、网络、输入只作为服务或通道
单向跨线程通信	输入线程生产消息，ECS 线程消费消息；ECS 生产同步包，网络线程消费发送
只读属性语义	`public T Service { get; }` 只限制引用不可重新赋值，不限制对象自身操作
批量消费优先	使用 `DequeueAll` 一次性取出整帧消息，避免循环 `TryDequeue`

IGameContext 扩展流程

`IGameContext` 是 ECS 系统访问全局共享服务的统一入口。

当前接口

`Gbar.Core/ECS/Interface/IGameContext.cs`

```
public interface IGameContext
{
    /// <summary>
    /// 输入消息队列，由 InputThread 生产，ECS 系统组在逻辑帧中消费。
    /// </summary>
    IMessageQueue<InputMessage> InputMessageQueue { get; }
}
```

如何新增全局服务

以新增 `IPhysicsWorld` 为例，按以下四步操作：

1. 在接口中声明新成员

```
public interface IGameContext
{
    IMessageQueue<InputMessage> InputMessageQueue { get; }
    IPhysicsWorld PhysicsWorld { get; }   // 新增
}
```

2. 在实现类中添加成员与构造函数参数

`Gbar.Server/GameThread/Threads/ArchThread.cs`

```
public class ArchThread : BaseGameThread, IGameContext
{
    public IMessageQueue<InputMessage> InputMessageQueue { get; }
    public IPhysicsWorld PhysicsWorld { get; }   // 新增

    public ArchThread(
        IMessageQueue<InputMessage> inputMessageQueue,
        IPhysicsWorld physicsWorld)              // 新增
    {
        InputMessageQueue = inputMessageQueue;
        PhysicsWorld = physicsWorld;
        TicksPerSecond = 60;
    }
}
```

3. 在 Server 入口注入实例

`Gbar.Server/Server.cs`

```
var messageQueue = new ConcurrentMessageQueue<InputMessage>();
var physicsWorld = new PhysicsWorld();        // 新增

var inputThread = new InputThread(messageQueue);
var archThread = new ArchThread(messageQueue, physicsWorld); // 新增
```

4. 在 System 中通过 context 使用

```
public void Update(float deltaTime)
{
    context.InputMessageQueue.DequeueAll(_messages);

    foreach (var msg in _messages)
    {
        // 处理输入
    }

    // 或使用新增服务
    // var result = context.PhysicsWorld.SweepSphere(...);
}
```

消息队列使用约定

只读属性 ≠ 只读队列

```
public IMessageQueue<InputMessage> MessageQueue { get; }
```

`get;` 仅表示**属性引用不可重新赋值**，队列实例本身仍支持 `Enqueue`、`TryDequeue`、`DequeueAll`。

生产与消费分工

线程/角色	操作	说明
`InputThread`	`Enqueue`	读取原始输入，解析为 `InputMessage` 后入队
ECS System	`DequeueAll` / `TryDequeue`	在逻辑帧中消费消息

推荐消费方式

优先使用 `DequeueAll` 批量读取，减少每帧多次出队开销：

```
private readonly List<InputMessage> _messages = [];

public void Update(float deltaTime)
{
    context.InputMessageQueue.DequeueAll(_messages);

    foreach (var msg in _messages)
    {
        // 处理消息
    }
}
```

系统生命周期

实现 `ISystem` 的系统由 `SystemGroup` 统一驱动。

```
Start(world, context)
       |
       ▼
Update(deltaTime)  ← 每帧循环
       |
       ▼
Destroy()
```

阶段	用途
`Start`	获取 `World` 与 `IGameContext` 引用，初始化查询器与缓存
`Update`	执行每帧逻辑，读取或修改组件数据
`Destroy`	释放资源，清理缓存

ECS 组件索引

> 位置： `Gbar.Core/ECS/Components/`

空间组件

组件	文件	说明
`Transform`	`Transform.cs`	`Position` + `Forward` 的组合组件，可拆开独立查询
`Position`	`Transform.cs`	世界坐标 `Vector3`，可独立附加到实体
`Forward`	`Transform.cs`	归一化朝向向量，提供 `Right()` 与 `Up()`
`Vector3`	`Transform.cs`	基础三维向量，提供 `Length`、`Normalized()`、`Cross`

属性组件

组件	文件	说明
`BoundedInt`	`Type/BoundedInt.cs`	通用当前值/最大值类型，自动 clamp，提供 `Add` / `Reduce` / `WithMax`
`Health`	`Health.cs`	生命值，代理 `BoundedInt`，提供 `IsDead`
`Mana`	`Mana.cs`	魔法值，代理 `BoundedInt`，提供 `IsEmpty` / `IsFull`
`Stats`	`Stats.cs`	基础战斗属性：力量/敏捷/智力/攻击/防御/移动速度

状态组件

组件	文件	说明
`State`	`State.cs`	原子化状态组件，值为 `EntityState`
`EntityState`	`EntityState.cs`	枚举：`Idle`、`Moving`、`Attacking`、`Casting`、`Stunned`、`Dead`

已移除组件

组件	移除原因
`Velocity`	位置更新由系统直接计算，不需要独立速度组件

组件	移除原因
`Status`	拆分为原子化 `Health` + `Mana` + `State`

查询示例

```
// 移动系统
var moveQuery = new QueryDescription().WithAll<Position, Forward>();

// 扣血：必须重新赋值
world.Query(in query, (ref Health hp) => {
    hp = hp.Reduce(10);
});

// 死亡判定
var deadQuery = new QueryDescription().WithAll<Health>();
world.Query(in deadQuery, (ref Health hp) => {
    if (hp.IsDead) { /* ... */ }
});

// 状态机切换
var stateQuery = new QueryDescription().WithAll<State>();
```

相关文档

文档	内容
tip/ecs-system-architecture.md	ECS System 架构设计、Group 执行顺序
tip/ecs-physics-design.md	ECS 与物理世界交互设计
tip/component-checklist.md	ECS 组件清单与优先级
tip/EventHandle.md	事件处理与跨线程通信速查